



Quran Companion Bot

Complete Deployment & Technical Guide

Built by Ayan Wajid | June 2026

■ Table of Contents

1.	Project Overview	3
2.	Architecture & How It Works	3
3.	Tech Stack	4
4.	Prerequisites	4
5.	Setting Up the Development Environment	5
6.	Building the Quran RAG System	6
7.	Setting Up Meta WhatsApp Business API	7
8.	Deploying to Railway	10
9.	Connecting Meta to Railway	12
10.	Getting a Permanent WhatsApp Token	13
11.	Environment Variables Reference	14
12.	Challenges & Solutions	14
13.	Prayer Reminder Timings	15
14.	Testing the Bot	15

■ 1. Project Overview

The Quran Companion Bot is a WhatsApp-integrated AI assistant built to help Muslims stay connected with the Quran in their daily lives. Instead of searching for verses manually, you simply send a message describing how you feel or what topic you need guidance on, and the bot finds the most relevant Quranic verse using semantic search across all 6236 ayahs.

Key Features

- Semantic verse search — understands meaning, not just keywords
- Exact surah/ayah lookup by name or reference number like 2:255
- General Islamic Q&A; with honest sourcing
- Off-topic filtering — won't hallucinate answers
- 5 daily prayer time reminders with random verses
- Bilingual support — responds in Urdu or English
- Permanent 24/7 hosting on Railway

■ 2. Architecture & How It Works

The bot uses a RAG (Retrieval Augmented Generation) architecture. This means the AI model never relies on its own memory for Quranic verses — it always retrieves verified text from a local vector database before responding.

Message Flow

Step	What Happens
1	You send a WhatsApp message to the bot number
2	Meta's servers receive it and POST it to your Railway webhook URL
3	Flask receives the POST and extracts your message text
4	The message is checked: casual greeting, exact lookup, or topic search
5	For topic search: message is embedded into a vector and ChromaDB finds the 3 closest verses
6	The verses + your message are sent to Groq (Llama 3.3 70B) as a prompt
7	Groq generates a warm, contextual response picking the best verse
8	The reply is sent back to your WhatsApp via Meta's API

■ 3. Tech Stack

Component	Technology	Why
Webhook server	Python + Flask	Lightweight, easy to deploy
AI model	Groq Llama 3.3 70B	Free, fast, strong reasoning
Vector database	ChromaDB	Local, no extra cost, persistent
Embeddings	all-MiniLM-L6-v2	Fast, accurate, runs locally
Quran dataset	quran-json (Sahih Int.)	Trusted, verified translation
WhatsApp API	Meta Cloud API	Official, free tier available
Hosting	Railway	Free tier, no SSL restrictions
Version control	GitHub	Required for Railway deploy

■ 4. Prerequisites

Before starting, make sure you have the following:

- Python 3.10 or higher installed on your machine
- Git installed (<https://git-scm.com/download/win>)
- A Facebook account (for Meta Developer Portal)
- A GitHub account (<https://github.com>)
- A Railway account (<https://railway.app>) — sign up with GitHub
- A Groq account (<https://console.groq.com>) — free
- VS Code or any code editor
- A phone with WhatsApp installed for testing

■ 5. Setting Up the Development Environment

Create Project Folder

```
mkdir quran-bot
cd quran-bot
```

Create Virtual Environment

```
python -m venv venv
```

Activate it:

On Windows:

```
venv\Scripts\activate
```

On Mac/Linux:

```
source venv/bin/activate
```

■ You will see (venv) at the start of your terminal when activated. Run the activate command every time you reopen the terminal.

Install Dependencies

Create a requirements.txt file with this content:

```
flask
requests
python-dotenv
groq
chromadb
sentence-transformers
schedule
unicorn
```

Then install:

```
pip install -r requirements.txt
```

Create .env File

Create a file called .env in your project folder:

```
WHATSAPP_TOKEN=your_token_here
PHONE_NUMBER_ID=your_phone_number_id
WHATSAPP_VERIFY_TOKEN=myquranbot123
GROQ_API_KEY=your_groq_key
YOUR_WHATSAPP_NUMBER=923001234567
```

■ Never share this file or push it to GitHub. Add .env to your .gitignore file.

■ 6. Building the Quran RAG System

Download the Quran Dataset

Download the Quran JSON from this URL and save as quran_en.json in your project folder:

```
https://cdn.jsdelivr.net/npm/quran-json@3.1.2/dist/quran_en.json
```

■ *This dataset contains all 114 surahs with Arabic text and Sahih International English translation. It is sourced from tanzil.net — a trusted Islamic resource.*

Create quran_rag.py

This file handles all verse retrieval. It contains three main components: the Surah name-to-number map (114 surahs), the Islamic term expander (Arabic terms to English), and the search_verse function which routes between exact lookup and semantic search.

Build the Vector Index (Run Once)

Create build_index.py and run it once:

```
python build_index.py
```

This script loads all 6236 verses, converts each English translation to a vector using the all-MiniLM-L6-v2 model, and stores them in ChromaDB. It takes 5-15 minutes on first run and creates a quran_index/ folder. Never delete this folder.

■ *The embedding model downloads automatically on first run (~90MB). After that it runs locally with no API calls needed.*

■ 7. Setting Up Meta WhatsApp Business API

■ Meta's interface changes frequently. The steps below reflect the UI as of June 2026.

Step 7.1 — Create a Meta Business Account

- Go to <https://business.facebook.com>
- Click Create Account if you don't have one
- Fill in any business name (your own name works fine)
- Note: New Facebook accounts may see a message saying to wait an hour before creating a business account. This is normal.

Step 7.2 — Create a Developer App

- Go to <https://developers.facebook.com>
- Click My Apps (top right) then Create App
- Enter app name and contact email, click Next
- On Use Cases screen, click Business messaging (3) on the left filter
- Select Connect with customers through WhatsApp
- Click Next through Requirements and Overview screens
- Click Create App and confirm with your Facebook password

Step 7.3 — Claim a Test Number

- In your app dashboard, click Use cases in the left sidebar
- Click Customize on the WhatsApp row
- Click Step 1. Try it out
- Click Claim test number — this gives you a free +1 555 number
- Note your Phone Number ID shown below the test number
- Click Generate token and select both `whatsapp_business_messaging` AND `whatsapp_business_management`
- Copy the token — this is your temporary `WHATSAPP_TOKEN` (expires every 24 hours)

■ *For a permanent token that never expires, see Section 10.*

Step 7.4 — Add Your Number as Test Recipient

- On the Step 1 Try it out page, scroll to Send a message from your test number
- Click the Recipient dropdown and add your personal WhatsApp number
- Include country code with no + or spaces (e.g. 923001234567 for Pakistan)
- WhatsApp will send a verification — confirm from your phone

■ 8. Deploying to Railway

Step 8.1 — Create Dockerfile

Create a Dockerfile in your project root:

```
FROM python:3.10-slim

WORKDIR /app

RUN apt-get update && apt-get install -y \
    build-essential \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
RUN pip install groq

COPY . .

EXPOSE 7860

CMD ["python", "app.py"]
```

Step 8.2 — Push to GitHub

Initialize git and push your project:

```
git init
git add app.py quran_rag.py build_index.py quran_en.json
git add requirements.txt Dockerfile .gitignore
git add quran_index/
git commit -m "quran bot"
git remote add origin https://USERNAME:TOKEN@github.com/USERNAME/quran-bot.git
git push origin master
```

■ Do NOT push your .env file. Add it to .gitignore before committing.

Step 8.3 — Create Railway Service

- Go to <https://railway.app> and sign in with GitHub
- Click New then New Web Service
- Click GitHub and connect your account
- Select your quran-bot repository
- Set Language to Docker and Branch to master
- Scroll down and set Instance Type to Free
- Click Deploy Web Service

Step 8.4 — Add Environment Variables

After the service is created, click on it and go to the Variables tab. Add all 5 variables:

Variable	Value
WHATSAPP_TOKEN	Your Meta access token
PHONE_NUMBER_ID	From Meta dashboard (e.g. 1129535900248614)
WHATSAPP_VERIFY_TOKEN	myquranbot123 (or any string you choose)
GROQ_API_KEY	From console.groq.com
YOUR_WHATSAPP_NUMBER	Your number e.g. 923001234567

■ After adding variables, you MUST click Deploy to apply them. Railway saves variables but does not restart automatically.

Step 8.5 — Get Your Railway URL

- Click on your service then Settings
- Click Networking on the right side
- Click Generate Domain
- Your URL will look like: `https://quran-bot-production-xxxx.up.railway.app`

■ 9. Connecting Meta to Railway

Step 9.1 — Configure Webhook

- Go to Meta Developer Portal, your app, Use cases, Customize
- Click Step 2. Production setup
- Expand Configure Webhooks
- Enter Callback URL: `https://YOUR-RAILWAY-URL/webhook`
- Enter Verify token: `myquranbot123` (must match `WHATSAPP_VERIFY_TOKEN`)
- Click Verify and save — should show green checkmark

Step 9.2 — Subscribe to Messages

Scroll down on the same page to find the Webhook fields list. Make sure the messages toggle is turned on (blue). This tells Meta to forward incoming WhatsApp messages to your webhook.

Step 9.3 — Subscribe Phone Number to App

Run this command to link your phone number to the webhook subscription:

```
curl -X POST \  
"https://graph.facebook.com/v19.0/YOUR_PHONE_NUMBER_ID/subscribed_apps" \  
-H "Authorization: Bearer YOUR_WHATSAPP_TOKEN" \  
-H "Content-Type: application/json"
```

You should get back: `{"success": true}`

■ *This step is often missed and is the most common reason messages don't get forwarded to your server.*

Step 9.4 — Publish the App

- Go to your app dashboard and click Publish in the left sidebar
- Add a Privacy Policy URL (your Railway URL works)
- Click Publish
- Status should change from Unpublished to Published

■ 10. Getting a Permanent WhatsApp Token

Meta's temporary access tokens expire every 24 hours. To avoid updating the token daily, create a System User with a permanent token.

- Go to <https://business.facebook.com>, then Settings
- In the left sidebar, click System users under Users
- Click Add and name it (e.g. Quranbot), set role to Admin
- After creating, click the ... button next to Revoke tokens
- Click Assign assets, choose Apps, select your app, give Full control
- Now click Generate token on the system user
- Select your app, check both whatsapp_business_messaging and whatsapp_business_management
- Set expiry to Never
- Copy the token and update WHATSAPP_TOKEN in Railway variables
- Click Deploy in Railway to apply the change

■ *This token never expires and survives page refreshes. You only need to do this once.*

■ 11. Environment Variables Reference

Variable	Where to Get It	Notes
WHATSAPP_TOKEN	Meta > System Users > Generate token	Use permanent token, not temporary
PHONE_NUMBER_ID	Meta > Step 1 Try it out > below test number	Numeric ID, not the phone number itself
WHATSAPP_VERIFY_TOKEN	You make this up	Must match what you enter in Meta webhook form
GROQ_API_KEY	console.groq.com > API Keys	Free, no regional restrictions
YOUR_WHATSAPP_NUMBER	Your personal number	Format: country code + number, no + or spaces

■ 12. Challenges & Solutions

Gemini API Regional Restriction

Problem

Pakistan's free tier for Gemini had limit: 0 — no requests allowed at all, not even with a fresh API key.

Solution

Switched to Groq which has generous free tier with no regional restrictions. Groq's Llama 3.3 70B is actually comparable in quality to Gemini Flash for this use case.

Keyword Search Was Too Limited

Problem

The original keyword-to-verse map only covered ~40 topics. Users asking about zina, specific emotions, or nuanced Islamic concepts got wrong or unrelated verses.

Solution

Replaced entirely with a RAG system. All 6236 verses are embedded in ChromaDB. Semantic search finds the right verse by meaning, not keywords. Added query expansion for Arabic/Islamic terms.

Meta Webhook Not Forwarding Messages

Problem

Even after setting up the webhook correctly, Meta was not forwarding incoming WhatsApp messages to our server. The webhook showed as verified but POST requests never arrived.

Solution

The missing step was calling the `subscribed_apps` endpoint via curl to explicitly subscribe the phone number to the webhook. This is not documented prominently in Meta's new UI.

Railway Environment Variables Not Applied

Problem

Added all 5 environment variables in Railway but the app kept crashing with `GROQ_API_KEY` not found errors.

Solution

Railway saves variables but does not restart automatically. You must click Deploy after adding or changing variables for them to take effect.

Hugging Face SSL Restriction

Problem

Deployed successfully to Hugging Face Spaces but outbound HTTPS connections to `graph.facebook.com` were blocked with SSL unknown error.

Solution

Migrated to Railway which has no such restrictions. Hugging Face free tier blocks outbound connections to external APIs.

WhatsApp Token Expiry

Problem

The temporary access token from Meta's Step 1 Try it out expires every 24 hours, requiring manual regeneration daily.

Solution

Created a System User in Meta Business Manager and generated a permanent token with Never expiry. This token survives indefinitely.

surah in Casual Trigger Match

Problem

The word muddaththir (a surah name) was being matched by the casual trigger sure causing the bot to treat surah lookup requests as casual greetings.

Solution

Changed the casual message detection from substring search to whole-word matching by splitting the message into words first before checking triggers.

■ 13. Prayer Reminder Timings

Railway servers run on UTC time. Pakistan Standard Time is UTC+5. To set a reminder for 5:00 AM Pakistan time, set the scheduler to 00:00 UTC.

Prayer	Pakistan Time (PKT)	UTC Time (Railway)
Fajr	5:00 AM	00:00
Dhuhr	1:00 PM	08:00
Asr	4:30 PM	11:30
Maghrib	6:45 PM	13:45
Isha	9:00 PM	16:00

■ *Adjust these times to match actual prayer times in your city. Rawalpindi/Islamabad times may vary by a few minutes seasonally.*

■ 14. Testing the Bot

Test Messages to Try

Message	Expected Behaviour
hi	Warm casual greeting, no verse
assalamualaikum	Islamic greeting response
I feel sad	Relevant verse about hardship/patience with explanation
I am stressed	Verse about reliance on Allah
tell me about zina	Surah 17:32 with Islamic context
give me surah fatiha	Full text of Al-Fatiha
2:255	Ayatul Kursi exact text
give me verse 94:5	For indeed with hardship will be ease
what is 2+2	Polite redirect to Islamic topics
how to code in python	Polite redirect to Islamic topics
jazakallah	Warm response, wa iyyakum

Checking Logs

Railway shows real-time logs. Look for:

- POST /webhook HTTP/1.1 200 — message received and processed successfully
- Message from XXXXXXXXXX: hi — confirms your message arrived
- Reply sent to XXXXXXXXXX — confirms reply was sent back
- Any Traceback — indicates an error to investigate

Built with the intention of making the Quran more accessible in daily life.

May it be of benefit. ■